P3 · 멀티 에이전트

# SubAgent vs Agent Team

## 단일은 1인 주방, 멀티는 역할형 주방 — 차이는 통신이다

2026년 2월 출시된 Agent Team의 핵심은 통신 — 팀원이 직접 메시지를 주고받고 공유 작업목록에서 과제를 가져간다.
강력한 만큼 비용도 크다. 서브에이전트 먼저, 협업이 필요할 때 팀이 기본기다.

# 이 파트의 세 가지 메시지

사람이 늘면 인건비가 늘듯, 에이전트가 늘면 토큰이 는다

**1**

## 차이는 통신이다

서브에이전트는 결과만 보고, 팀원은 서로 직접 메시지를 주고받는다.

**2**

## 강력한 만큼 비싸다

팀원마다 독립 컨텍스트(최대 100만 토큰) — 3인 팀은 보통 단일의 3~4배.

**3**

## 서브에이전트 먼저

빠른 워커는 서브에이전트, 협업·교차검증이 필요할 때 팀으로 올린다.

**2026.02 출시 — 여러 세션이 직접 통신하며 협업하는 길이 열렸다**

# 1인 주방 vs 역할형 주방

## 단일 에이전트 · 1인 주방

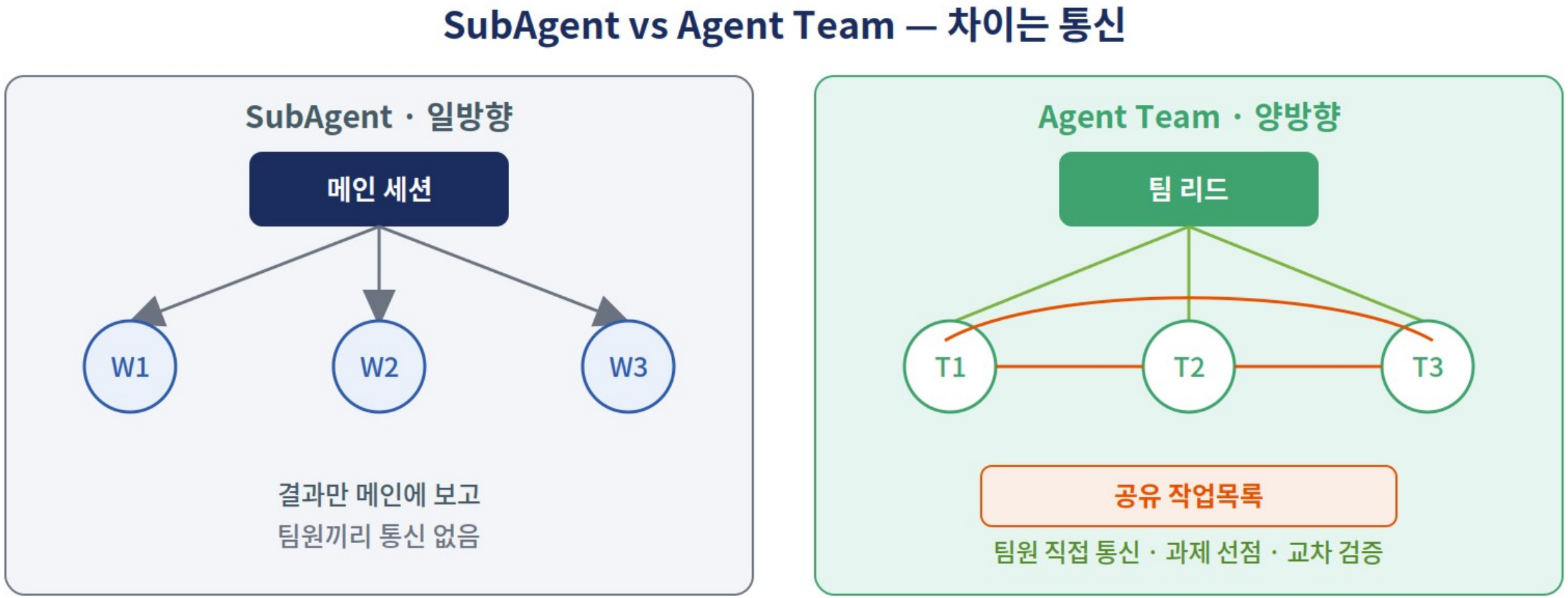
- 한 명이 주문·조리·검수를 모두
- 단순한 일에 빠르고 효율적
- 복잡한 코스에서는 병목
- 토큰 비용이 가장 낮다

## 멀티 에이전트 · 역할형 주방

- 역할을 나눠 동시에 일한다
- 복잡·대량 작업에서 빨라진다
- 협업·교차검증으로 품질↑
- 에이전트 수만큼 토큰 비용↑

# SubAgent vs Agent Team

허브(일방향)와 메시(양방향) — 통신 구조가 비용과 적합성을 가른다



# 일방향 위임과 양방향 협업

## SubAgent — 일방향 위임

- 별도 컨텍스트 창에서 도는 워커
- 리드가 보내고 → 실행 → 결과만 회수
- 형제끼리 공유 상태·직접 통신 없음
- 빠르고 반복적인 작업에 적합

## Agent Team — 양방향 협업

- 각 팀원이 독립 컨텍스트 창에서 일함
- peer-to-peer 메시지 + 공유 작업목록
- 동시 쓰기를 막는 파일 잠금
- 잘못된 중간 결론이 번지지 않게 격리

# settings.json로 켠다

실험 기능 — 환경 변수에 기능을 명시하면 켜진다

## settings.json로 켜고, 한 문장으로 구성한다

**활성화 — ~/.claude/settings.json**

```
{
 "env": {
  "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
 }
}

// Claude Code v2.1.32 이상 필요
```

**구성 — 평문 한 문장**

결제 모듈을 리팩터링할
에이전트 팀을 구성하라.

팀원 셋을 스폰한다 —
API 계층 · DB 마이그레이션 ·
테스트 커버리지.

공유 작업목록으로 조율하게 하라.

팀 리드 + 독립 팀원 셋을 스폰해, 공유 작업목록과 직접 메시지로 조율한다

# 토큰 비용의 경제학

비용은 팀원 수와 작업량에 비례한다 — 그러나 규모가 크면 역설이 생긴다

| 파이프라인 규모 | 권장 | 비용 특성 |
| --- | --- | --- |
| 소규모 (2~4) | SubAgent | 비용 차이 미미, 가장 효율적 |
| 중규모 (5~10) | 작업에 따라 | 서브에이전트 오케스트레이터가 30~50k 토큰까지 누적 |
| 대규모 (10+ 병렬) | Agent Team | 결과가 한 곳에 몰리지 않아 팀이 3~5배 저렴할 수 있음 |

4인 팀은 컨텍스트를 4번 초기화하는 비용을 선불로 치르고, 메시지마다 왕복이 더해진다.

# 비용을 줄이고 컨텍스트를 지킨다

## 비용을 줄이는 4수칙

- 모델 티어링 — 복잡은 상위, 단순은 경량
- 유휴 에이전트 즉시 종료 (/stop)
- 디스패치 사이 /clear — 토큰 30~50% 절감
- YAML에 모델 상한을 고정해 레포에 커밋

## 컨텍스트 관리 = 재료 보관

- 필요한 정보만 작업대에 올린다
- context rot — 길수록 회수 정확도↓
- autocompact는 약 187K 토큰에서 발동
- 메모리·컴팩션·도구 결과 비우기로 대응

# 무엇을 언제 쓰는가

기본기는 '서브에이전트 먼저'

## 1 서브에이전트 먼저

빠르고 집중된 워커가 결과만 보고하면 될 때 쓴다.

## 2 팀으로 올린다

발견 공유·가정 반박·자율 조율이 필요한 복잡 작업일 때.

## 3 계층적으로 접근

먼저 역할과 경계를 정하고, 그다음 실행을 위한 병렬 팀.

**대규모 병렬에서 팀이 더 싼 이유: 결과가 한 컨텍스트로 몰리지 않는다**

# 두 개의 토론 질문

## Q1 허브와 메시

'여러 사람이 서로 결과를 봐야 빨라지는' 일과 '각자 따로 해도 되는' 일을 하나씩 꼽아 보자. 각각 메시(팀)와 허브(서브에이전트) 중 어디에 맞는가?

## Q2 비용의 손익분기

대규모 병렬 작업에서 팀이 오히려 더 쌀 수 있는 이유를 한 문장으로 설명해 보자.
본인의 가장 비싼 워크플로우에 이 원리를 적용한다면?

# 한 줄 정리

**"서브에이전트는 허브, 에이전트 팀은 메시 — 차이는 통신이고, 통신이 비용을 가른다."**

**다음: P4 — 메타 하네스 (revfactory/harness)**

지금까지는 하네스를 손으로 짰다. 다음은 자동 생성이다.